Google Workspace Admin SDK & API Automation / Reporting Toolkit

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

- What this is . . . 4
- How it works . . . 5
- Reference model . . . 5
- Benefits . . . 6
- Limitations . . . 6
- Deployment model . . . 6

Requirements 7

- 1. Google Workspace super-admin (for one-time delegation setup only) . . . 7
- 2. A dedicated Google Cloud project for automation . . . 7
- 3. Automation service identity with domain-wide delegation . . . 7
- 4. No exported service-account key committed anywhere . . . 8
- 5. Python 3.10+ toolchain with the Google client libraries . . . 8
- 6. Least-privilege admin roles and OAuth scope inventory . . . 9

Global Requirements — Google Workspace & Gemini Solutions 9

- 1. Google Workspace / Cloud Identity tenant . . . 9
- 2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10
- 3. DNS control for the routed domains . . . 10
- 4. Identity & provisioning source of truth . . . 10
- 5. A pilot scope (OU / group) and change control . . . 10
- 6. Security, compliance & governance stakeholders . . . 11

Implementation 11

- Stage 1 — Assess . . . 11
- Stage 2 — Design . . . 11
- Stage 3 — Configure . . . 12
- Stage 4 — Pilot . . . 13
- Stage 5 — Validate . . . 14
- Stage 6 — Expand . . . 14

Testing & Validation 14

- Test 1 — Auth least-scope (Gate 3-A) . . . 14
- Test 2 — Discovery baseline is read-only and complete . . . 15
- Test 3 — Reports accurate (Gate 4-A) . . . 15
- Test 4 — Automation idempotent, dry-run (Gate 4-B) . . . 15
- Test 5 — Apply correctness (Gate 4-C) . . . 15

Test 6 — Rollback works . . . 15
Test 7 — No secrets in code / no key committed . . . 16
Test 8 — Scheduled evidence (Gate 6-A) . . . 16
Troubleshooting 16
Authentication & delegation . . . 16
Quota, throttling & performance . . . 17
Reports API specifics . . . 17
Drive sharing report . . . 17
Provisioning . . . 18
Scheduling . . . 18
Best Practices 18
Security & credentials . . . 19
Safe automation . . . 19
Reliable API access . . . 19
Reporting & evidence . . . 20
Maintainability . . . 20
Deliverables Summary 20
Documentation . . . 20
The 5 Python tools . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary

Outcome. The organization gains a secure, least-privilege automation and reporting toolkit for Google Workspace, built on the **Admin SDK and REST APIs** (Directory, Reports, Drive, Groups) and driven by **Python**. Routine administration (user / OU / license provisioning) and evidence (external-sharing exposure, login and admin audit, group membership) become **automated, repeatable, and auditable** — replacing manual console work and screenshot "reports" that do not scale in a tenant of tens of thousands of users.

Approach. A dedicated Google Cloud project hosts a purpose-built **service identity** with **domain-wide delegation** authorized for an **explicit, minimal OAuth scope list** and impersonating a **least-privilege** admin — never a super-admin for routine reads. A shared auth + API-client layer gives every tool consistent least-scope authentication, pagination, and backoff. On top sit reusable patterns: idempotent provisioning, sharing and audit reporting, and a scheduling model that emits the exact cron / Cloud Scheduler / Task Scheduler definitions to run reports on a cadence. Delivery follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback.

Why it is low-risk. Every tool is **safe by default**: discovery and reporting are read-only, and the provisioning tool defaults to **dry-run**, printing exactly what it would change and only mutating when an operator passes `--apply`. Apply runs write a **reversible change log** enabling one-command rollback, contained to the pilot OU. **No credentials, keys, or secrets ever live in code** — authentication uses ADC / a secured credential / a secrets manager, with the narrowest scope each task requires. Validation gates prove least-scope auth, report accuracy, provisioning idempotency, and scheduled evidence before any expansion.

Deliverables. Full implementation, requirements, testing, troubleshooting, and best-practice documentation, plus **five authentic Python tools** (auth/scope validator, tenant discovery, idempotent provisioning, sharing/audit report generator, and a scheduling/config helper) the organization can run to implement and operate the toolkit itself — at a fraction of the cost of bespoke consulting.

Solution Overview

What this is

An enterprise organization running Google Workspace at scale (tens of thousands of users) cannot administer or evidence its tenant with the Admin console and manual screenshots alone. Provisioning, license moves, sharing reviews, and audit "reports" pasted from console screens do not scale, are not repeatable, and are not defensible to auditors.

This solution builds an **Admin SDK & API automation and reporting toolkit**: a secure service identity, least-scope authenticated Python against the Google Workspace **Directory**, **Reports**, **Drive**, and **Groups** APIs, a set of reusable automation and reporting patterns (provisioning, license, sharing, login/audit reports), and a scheduling model — so that administration and evidence become automated, repeatable, and auditable.

It is deliberately **distinct** from an Apps Script / AppSheet governance solution. This toolkit is about **building** first-party administrative automation and reporting against Google's Admin SDK and REST APIs

from an external Python runtime, using a properly configured service identity — not about governing citizen-developer platforms.

The toolkit is **safe by default**. Every discovery and reporting tool is read-only; every tool that can change the tenant defaults to `--dry-run` and only mutates when the operator passes `--apply`. **No credentials, keys, or secrets are ever written into code.** Authentication uses OAuth, a secured service-account credential, or Application Default Credentials (ADC), with the **narrowest** OAuth scopes each task requires.

How it works

The toolkit has five layers.

1. **Secure service identity.** A dedicated Google Cloud project holds a **service account** used for

automation. For APIs that act on user data (Directory, Reports, Drive, Groups) the service account is granted **domain-wide delegation (DWD)** in the Admin console, authorized for an **explicit, minimal list of OAuth scopes** and impersonating a **least-privilege admin** subject — never a super-admin for routine reads. Alternatively, an OAuth client (installed-app / user consent) is used for interactive, per-admin operations. The service-account **key is never committed**; credentials come from ADC, a secrets manager, or Workload Identity where the runtime supports it.

1. **Least-scope authentication.** A single auth helper centralizes credential acquisition and scope

selection so every tool requests only the scopes it needs — for example `admin.directory.user.readonly` for a user report versus `admin.directory.user` only for a provisioning tool run with `--apply`. Read-only scopes are the default; write scopes are opt-in.

1. **Authenticated API access.** A thin, reusable API-client layer wraps the Google API Python client

(`google-api-python-client`) with automatic **pagination**, **exponential-backoff retry** on 429 / 5xx responses, and consistent error handling — so every tool talks to Directory / Reports / Drive / Groups the same way.

1. **Reusable automation & reporting patterns.** On top of the client sit the patterns the toolkit

ships: user / OU / license **provisioning** (idempotent, dry-run first), **Drive sharing exposure** reporting, **login / admin audit** reporting from the Reports API, and **group membership / configuration** reporting — each producing a named, timestamped artifact (CSV/JSON) suitable for audit evidence.

1. **Scheduling model.** A scheduling/config helper renders the exact command lines, cron / Cloud

Scheduler / Task Scheduler entries, and least-scope service-identity requirements needed to run each report on a cadence — so recurring evidence is produced automatically and consistently.

Reference model

The toolkit is built and operated to a single naming convention. Every artifact it creates is named `<Purpose> – <Scope> – <Stage>` and that exact name is reused in testing, troubleshooting, and the scripts:

Artifact	Name (this solution)
Automation Google Cloud project	`Automation Project – Workspace – Core`
Automation service account	`Automation SA – AdminSDK – Core`

Artifact	Name (this solution)
Domain-wide-delegation grant	`DWD Grant – AdminSDK – LeastScope`
Least-privilege impersonation subject	`Impersonation Subject – Reports – ReadOnly`
Discovery / assessment report	`Discovery Report – Tenant – Assess`
User/OU/license provisioning run	`Provisioning Run – Finance-OU – Pilot`
Drive sharing exposure report	`Sharing Report – External – Validate`
Login/admin audit report	`Audit Report – Login – Validate`
Scheduled report definition	`Schedule Definition – Reports – Expand`

Benefits

• **Repeatable, auditable administration** — provisioning and reports run identically every time and emit dated evidence files, replacing manual screenshots.

• **Least privilege by construction** — every task requests only the scopes it needs; write access is opt-in and dry-run first.

• **No secrets in code** — credentials come from ADC / a secured credential / a secrets manager; keys are never committed or exported into the repo.

• **Safe to run** — read-only or `--dry-run` by default; a destructive run requires an explicit `--apply` and prints exactly what it will change first.

• **Portable** — standard-library-first Python plus the official Google client; runs on any Python 3.10+ host, in a container, or in a scheduler.

Limitations

• The toolkit **automates and reports on** Workspace; it does not replace the Admin console for interactive, one-off, or preview-only settings.

• **Reports API data latency** varies by application; some activity events surface with a delay (**verify the current latency for your event types** in Google's documentation). Reports are point-in-time evidence, not real-time monitoring.

• Some Workspace/Gemini settings are **not exposed by a stable public API** (flag "verify in your tenant"); those remain console-managed and are out of scope for automation here.

• Domain-wide delegation is powerful; it must be scoped and reviewed as described in Best Practices.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback for any production change (provisioning writes especially). You stand up and validate the service identity and read-only reporting first, prove automation on a **pilot OU/group** in dry-run, then apply and expand.

Requirements

These are the prerequisites **specific to this toolkit**. They build on the shared category baseline in `global-requirements-google.md` (a provisioned Workspace/Cloud Identity tenant with a super-admin, a defined identity source of truth, a pilot OU/group, and change control) — verify those first, then the solution-specific items below. Each item lists Description, Why required, How to verify, and How to implement if missing.

1. Google Workspace super-admin (for one-time delegation setup only)

• **Description:** A Workspace **super-admin** account, used only to create the Google Cloud project, enable APIs, and authorize domain-wide delegation. Routine automation does **not** run as super-admin.

• **Why required:** Enabling Admin SDK APIs and adding a DWD grant with scopes are super-admin actions in the Admin console (**Security** → **Access and data control** → **API controls** → **Domain-wide delegation**).

• **How to verify:** Sign in to **admin.google.com**; confirm the account shows the **Super Admin** role under **Directory** → **Users** → **(account)** → **Admin roles and privileges**.

• **How to implement if missing:** Have an existing super-admin grant the role, or perform the one-time setup steps under an existing super-admin.

2. A dedicated Google Cloud project for automation

• **Description:** A Google Cloud project — `Automation Project – Workspace – Core` — that hosts the automation service identity and has the required APIs enabled: **Admin SDK API**, **Google Drive API**, and (if used) **Cloud Identity API**. The **Reports** and **Directory** capabilities are part of the Admin SDK API.

• **Why required:** API calls are billed/attributed to and quota-managed per project; keeping automation in its own project isolates it and simplifies audit.

• **How to verify:** In the Google Cloud console, **APIs & Services** → **Enabled APIs & services** lists the Admin SDK API and Drive API for the project.

• **How to implement if missing:** Create the project, then **enable** the Admin SDK API, Drive API, and Cloud Identity API from **APIs & Services** → **Library**.

3. Automation service identity with domain-wide delegation

• **Description:** A service account — `Automation SA – AdminSDK – Core` — with **domain-wide delegation**

authorized in the Admin console for an **explicit, minimal scope list** (`DWD Grant – AdminSDK – LeastScope`), impersonating a least-privilege admin subject (`Impersonation Subject – Reports – ReadOnly`) rather than a super-admin.

• **Why required:** Directory / Reports / Drive / Groups calls act on domain user data and require a

delegated identity; least-scope + least-privilege impersonation limits blast radius.

• **How to verify:** In the Admin console **API controls** → **Domain-wide delegation**, the service

account's **client ID** is listed with exactly the intended scopes (no broader wildcard). Run the toolkit's auth validator (see §5) — it confirms each scope resolves.

• **How to implement if missing:** Create the service account, note its **OAuth client ID**, then add the

DWD grant with the minimal scope list. Start with `*.readonly` scopes; add a write scope only for the provisioning tool.

4. No exported service-account key committed anywhere

• **Description:** Credentials are supplied at runtime via **Application Default Credentials (ADC)**, a

secrets manager, or **Workload Identity** — never a `.json` key file stored in the repo or baked into an image.

• **Why required:** A leaked service-account key with DWD is a domain-wide compromise. The toolkit's

security posture depends on keys never living in source control.

• **How to verify:** Repo scan finds no `*.json` key, no private-key material, and no credentials in

environment files committed to version control. The scripts read credentials only from ADC / a secured path passed at runtime.

• **How to implement if missing:** Configure ADC (`gcloud auth application-default login` for local dev,

or attached identity in cloud), or store the credential in a secrets manager and reference it by path at runtime. Remove and **rotate** any key that was previously committed.

5. Python 3.10+ toolchain with the Google client libraries

• **Description:** Python **3.10 or newer** with `google-api-python-client` and `google-auth` available

(a virtual environment is recommended). The toolkit is otherwise standard-library-first.

• **Why required:** The scripts call the Google APIs through the official client and use modern Python

typing/features.

• **How to verify:** `python --version` reports ≥ 3.10; `pip show google-api-python-client google-auth`

reports installed versions.

• **How to implement if missing:** Install Python 3.10+, create a venv, and `pip install google-api-python-client google-auth`.

6. Least-privilege admin roles and OAuth scope inventory

• **Description:** A documented mapping of **which task needs which scope** and **which impersonation subject** it uses — read-only scopes for reporting (`admin.directory.user.readonly`, `admin.reports.audit.readonly`, `drive.readonly`, `admin.directory.group.readonly`) and write scopes only where a provisioning tool is run with `--apply`.

• **Why required:** Least-scope is the core security control; it must be explicit and reviewed, not incidental.

• **How to verify:** The scope inventory exists and matches the DWD grant; the auth validator (script 1) reports each configured scope and flags any scope that is broader than the task requires.

• **How to implement if missing:** Build the inventory from the tool set below, grant only those scopes, and review it as part of change control whenever a new automation is added.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model;

the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance**

stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to

enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements

before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Follow these steps in order. The model is **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first. Every artifact is named `<Purpose> – <Scope> – <Stage>` and the same name is reused later in Testing, Troubleshooting, and the scripts. **No secrets are written into code** at any step; write operations are **dry-run first** and require an explicit `--apply`.

> *High-impact changes in this solution are the **provisioning writes** (`Provisioning Run – Finance-OU – Pilot` and later). Each such step names its **pass/fail gate** and its **rollback**.*

Stage 1 — Assess

1.1 Inventory what you will automate and report on

Confirm the scope: which OUs/groups you provision, which reports you must produce (user/license, external Drive sharing, login/admin audit, group membership), and the required cadence. Record the **scope inventory** from Requirements §6 — the task→scope→subject mapping.

1.2 Establish the read-only baseline

Nothing here changes the tenant. You will stand up read-only access first and only add write scopes for the pilot provisioning step.

Stage 2 — Design

2.1 Define the naming and artifact conventions

Adopt the reference-model names (Overview). The service identity is `Automation SA – AdminSDK – Core`; its delegation grant is `DWD Grant – AdminSDK – LeastScope`; its impersonation subject is `Impersonation Subject – Reports – ReadOnly`.

2.2 Design the scope model (least scope)

Assign the **narrowest** scope per task:

Task	Scope	Access
Discovery / user & license report	`admin.directory.user.readonly`	read
Group membership report	`admin.directory.group.readonly`	read
Login / admin audit report	`admin.reports.audit.readonly`	read
Drive external-sharing report	`drive.readonly` (or `drive.metadata.readonly`)	read
User/OU/license provisioning (`--apply`)	`admin.directory.user`	write

Read scopes are the default set on the DWD grant. The **write** scope is added only when the pilot provisioning step is authorized.

Stage 3 — Configure

3.1 Create the automation project and enable APIs

As a super-admin, in the Google Cloud console create the project `Automation Project – Workspace – Core`. Under **APIs & Services** → **Library**, enable: **Admin SDK API**, **Google Drive API**, and **Cloud Identity API**. (Directory and Reports are served by the Admin SDK API.)

3.2 Create the service identity

In **IAM & Admin** → **Service Accounts**, create `Automation SA – AdminSDK – Core`. **Do not create or download a JSON key.** Record its **OAuth client ID** (the numeric unique ID) for the delegation grant.

3.3 Authorize domain-wide delegation with least scope

In the **Admin console** → **Security** → **Access and data control** → **API controls** → **Domain-wide delegation**, add a new grant `DWD Grant – AdminSDK – LeastScope`:

- **Client ID:** the service account's OAuth client ID from 3.2.
- **OAuth scopes:** the **read-only** scopes from 2.2, comma-separated. Add

`admin.directory.user` (write) **only** when you authorize the pilot provisioning step (3.6).

3.4 Choose the impersonation subject

Create or designate a **least-privilege admin** account `Impersonation Subject – Reports – ReadOnly` (with only the admin privileges the reports require) as the subject the service account impersonates. Do **not** impersonate a super-admin for routine runs.

3.5 Configure runtime credentials (no keys in code)

Provide credentials via **ADC** (`gcloud auth application-default login` for local dev, or an attached identity in cloud), or reference a secured credential from a **secrets manager** at runtime. Confirm the scripts read credentials only from ADC / a path passed at runtime — never a committed file.

Gate 3-A (auth least-scope): run the auth validator (`gwsdk_auth_scope_validator.py`). **Pass** = it authenticates, lists exactly the intended scopes, and flags none as broader than required. **Fail** = fix the DWD grant / impersonation subject before proceeding. **Rollback:** none needed (read-only); remove any over-broad scope from the grant.

3.6 (Deferred) authorize the write scope for provisioning

Only when you reach the pilot provisioning step, add `admin.directory.user` to `DWD Grant – AdminSDK – LeastScope`. This is a **change-controlled** action.

Stage 4 — Pilot

4.1 Run discovery (read-only)

Run the discovery tool to produce `Discovery Report – Tenant – Assess` (user counts, OU distribution, license usage, suspended accounts). This is read-only and establishes the baseline.

4.2 Produce the read-only reports on the pilot scope

Generate, scoped to the pilot OU/group:

- `Sharing Report – External – Validate` (Drive external-sharing exposure), and
- `Audit Report – Login – Validate` (login / admin activity from the Reports API).

Confirm each writes a dated CSV/JSON artifact and the row counts are plausible.

Gate 4-A (reports accurate): spot-check a sample of rows against the Admin console for the same pilot users/files. **Pass** = sampled rows match the console. **Fail** = investigate scope / impersonation / API before trusting the report.

4.3 Dry-run the provisioning automation

Run the provisioning tool against the pilot OU `Provisioning Run – Finance-OU – Pilot` **without** `--apply` (dry-run is the default). It reads the desired-state input (CSV) and prints, per user, the exact create/update/move/license action it *would* take — **making no change**.

Gate 4-B (automation idempotent, dry-run): run the dry-run **twice**. **Pass** = the planned actions are identical and, for users already in the desired state, the plan shows **no change** (idempotent). **Fail** = fix the desired-state input or matching logic before applying.

4.4 Apply the provisioning on the pilot (change-controlled)

After Gate 4-B passes and the write scope is granted (3.6), re-run with `--apply` for the pilot OU only. The tool prints each change as it makes it and writes a result log.

Gate 4-C (apply correctness): re-run the tool in dry-run immediately after apply. **Pass** = the plan now shows **no changes** (the tenant matches desired state) and the audit report shows the expected admin actions. **Fail** → **rollback**.

> ***Rollback for provisioning (**`Provisioning Run – Finance-OU – Pilot`**):** the apply run writes a reversible **change log** (prior value per field per user). To roll back, feed that change log to the tool's `--revert <logfile>` mode (dry-run first, then `--apply`) to restore prior OU / license / attribute values, or suspend newly created accounts. Because writes are least-scope (`admin.directory.user`) and per-user, blast radius is contained to the pilot OU.*

Stage 5 — Validate

5.1 Confirm all gates green

Auth least-scope (3-A), reports accurate (4-A), automation idempotent (4-B), apply correctness (4-C). Re-run the full read-only report set and confirm artifacts are dated and complete. See `05-testing.md` for the detailed validation procedures.

Stage 6 — Expand

6.1 Broaden scope

Once the pilot is validated, expand the provisioning desired-state input and the report scope from the pilot OU/group to the next waves, one controlled step at a time — re-running Gate 4-B (dry-run idempotency) before each `--apply`.

6.2 Schedule the recurring reports

Run the scheduling helper to emit `Schedule Definition – Reports – Expand`: the exact command lines and cron / Cloud Scheduler / Task Scheduler entries for each recurring report, plus the least-scope service-identity requirements. Deploy those to your scheduler so evidence is produced automatically.

Gate 6-A (scheduled evidence): confirm the first scheduled run produces the expected dated artifact in the expected location. **Pass** = artifact present and complete. **Fail** = fix the schedule / credentials (see Troubleshooting).

Testing & Validation

Each procedure states the exact check and its **expected pass/fail** signal. Reuse the artifact names from Implementation. Run all read-only validations before any `--apply`.

Test 1 — Auth least-scope (Gate 3-A)

Procedure: Run `gwsdk_auth_scope_validator.py`. It authenticates the service identity (`Automation SA – AdminSDK – Core`) impersonating `Impersonation Subject – Reports – ReadOnly`, lists the scopes actually granted on `DWD Grant – AdminSDK – LeastScope`, and compares them to the declared task→scope inventory.

- **Pass:** authentication succeeds; the granted scopes exactly match the intended read-only set (plus the write scope only if provisioning is authorized); no scope is flagged as broader than required.
- **Fail:** authentication fails (`unauthorized_client` / `access_denied`) or an over-broad or missing

scope is reported. Fix the DWD grant / impersonation subject before continuing.

Test 2 — Discovery baseline is read-only and complete

Procedure: Run the discovery tool to produce `Discovery Report – Tenant – Assess`.

- **Pass:** a dated report is written with user counts, OU distribution, license usage, and suspended accounts; the run made **no** write calls (verify no admin activity from the run in the audit report).
- **Fail:** missing sections, an empty report, or any write call observed.

Test 3 — Reports accurate (Gate 4-A)

Procedure: For the pilot scope, generate `Sharing Report – External – Validate` and `Audit Report – Login – Validate`. Spot-check a sample of rows against the Admin console (same users/files/events).

- **Pass:** sampled rows match the console (external-shared files are genuinely external; login events match sign-in activity).
- **Fail:** discrepancies → check scope, impersonation subject, and Reports API event latency before trusting the report.

Test 4 — Automation idempotent, dry-run (Gate 4-B)

Procedure: Run `Provisioning Run – Finance-OU – Pilot` in **dry-run** (default) **twice** against the same desired-state input.

- **Pass:** both runs produce an identical plan; users already in desired state show **no change**.
- **Fail:** the two plans differ, or a no-op user shows a planned change → fix desired-state input or matching logic. Do **not** proceed to `--apply`.

Test 5 — Apply correctness (Gate 4-C)

Procedure: After `--apply` on the pilot OU, immediately re-run the provisioning tool in dry-run.

- **Pass:** the plan now shows **no changes** (tenant matches desired state); the audit report shows the expected admin actions and nothing more.
- **Fail:** residual planned changes or unexpected admin actions → **roll back** via the change log (`--revert <logfile>`), then investigate.

Test 6 — Rollback works

Procedure: In a controlled test, apply a small change on the pilot OU, then run `--revert <logfile>` (dry-run, then `--apply`).

- **Pass:** the revert plan exactly reverses the applied change and, after apply, the tenant is back to the prior state (dry-run shows no diff).

• **Fail:** the revert plan is incomplete → the change log is not capturing prior values correctly; fix before relying on any production apply.

Test 7 — No secrets in code / no key committed

Procedure: Scan the repo for private-key material, `*.json` service-account keys, and credentials in committed files; confirm the scripts obtain credentials only from ADC / a runtime-supplied path.

- **Pass:** no key or secret is present; credentials resolve from ADC / secrets manager only.
- **Fail:** any key or secret found → remove, **rotate** it, and reconfigure to ADC/secrets manager.

Test 8 — Scheduled evidence (Gate 6-A)

Procedure: Deploy `Schedule Definition – Reports – Expand` and let the first scheduled run execute.

- **Pass:** the expected dated artifact appears in the expected location, complete.
- **Fail:** missing/partial artifact → check the scheduler entry, the runtime credential (ADC available to the scheduled context), and the tool's exit code/log.

Troubleshooting

Common failure modes when building and running Admin SDK / API automation, with resolutions. Reuse the artifact names from Implementation.

Authentication & delegation

`unauthorized_client` / *"Client is unauthorized to retrieve access tokens using this method"*

- **Cause:** the service account's OAuth **client ID** is not on `DWD Grant – AdminSDK – LeastScope`, or a requested scope is not in the grant's scope list.
- **Resolve:** in the Admin console **API controls** → **Domain-wide delegation**, confirm the exact numeric client ID and that **every** scope the tool requests is listed (comma-separated, no typos). Scope changes can take a few minutes to propagate — re-run the auth validator after.

`access_denied` / *403* `Not Authorized to access this resource/api`

- **Cause:** the **impersonation subject** (`Impersonation Subject – Reports – ReadOnly`) lacks the admin privilege the API requires, or you are impersonating a non-admin.
- **Resolve:** impersonate a subject with the least admin privilege the report needs (e.g. a role that can read users/reports). Do not escalate to super-admin as a fix — grant the specific privilege.

`invalid_grant` / *"Invalid impersonation 'sub'"*

- **Cause:** the subject email is mistyped, not in the domain, or suspended.
- **Resolve:** verify the subject exists, is active, and is spelled correctly; pass it via `--subject`.

Credentials not found / ADC error

- **Cause:** no Application Default Credentials in the runtime context (common in scheduled contexts).
- **Resolve:** run `gcloud auth application-default login` for local dev, or attach an identity / Workload Identity in cloud, or point `--credentials` at a secured credential. **Never** commit a key to fix this.

Quota, throttling & performance

HTTP 429 `rateLimitExceeded` / `userRateLimitExceeded`

- **Cause:** exceeding per-project or per-user API quota, often on large Directory/Drive enumerations.
- **Resolve:** the tools already retry with exponential backoff (`--max-retries`); for sustained load, reduce concurrency, request a quota increase for `Automation Project – Workspace – Core`, or schedule large reports off-peak. Keep automation in its own project so its quota is isolated.

HTTP 500 / 503 (transient)

- **Cause:** transient Google-side errors.
- **Resolve:** covered by the built-in backoff; if persistent, retry later and check Google Workspace status.

Report runs slowly / times out on a large tenant

- **Cause:** enumerating all users/files without paging efficiently.
- **Resolve:** the tools paginate (500 users / 100 files per page); scope the report (e.g. by OU or a narrower Drive query) for the pilot, and run full-fleet reports on a schedule rather than interactively.

Reports API specifics

Login/admin events are missing or delayed

- **Cause:** Reports API activity events surface with **latency that varies by application/event type**.
- **Resolve:** treat reports as point-in-time evidence; widen `--days`, and **verify the current latency for your event types** in Google's documentation before concluding an event "didn't happen".

`Audit Report – Login – Validate` *returns empty*

- **Cause:** wrong `applicationName`, too-short look-back, or the subject lacks reports privilege.
- **Resolve:** confirm `--report login`, increase `--days`, and confirm the impersonation subject can read reports (see `access_denied` above).

Drive sharing report

`Sharing Report – External – Validate` *misses shared-drive files or over-/under-reports external*

- **Cause:** missing `includeItemsFromAllDrives` / `supportsAllDrives`, or the domain comparison is off.
- **Resolve:** the tool sets the all-drives flags and compares each permission's domain/address against `--domain`; confirm `--domain` is your **primary** domain and re-check. `anyone` links and other-domain users/domains are the external grants flagged.

Provisioning

Dry-run and apply plans differ / a NOOP user shows a change (Gate 4-B fails)

- **Cause:** the desired-state CSV has a value that never matches (whitespace, case, boolean format), so the tool always sees a diff.
- **Resolve:** normalize the CSV (trim, consistent `true/false`, exact `orgUnitPath`); the tool lowercases emails and parses common boolean spellings — align the input to those.

`CREATE` *action raised an error refusing to invent a password*

- **Cause:** the desired-state CSV references a user who does not exist; the tool intentionally refuses to fabricate account-creation credentials.
- **Resolve:** create the account through your onboarding process (which sets the initial password per policy), then re-run the tool to reconcile OU/attributes idempotently.

Applied a wrong change — need to roll back

- **Cause:** an incorrect desired-state input was applied.
- **Resolve:** run `gwsdk_provision_users.py --revert <change-log.json>` (dry-run first, then `--apply`) to restore prior values from the reversible change log written by the apply run.

Scheduling

Scheduled report produced nothing (Gate 6-A fails)

- **Cause:** the scheduled context lacks ADC, the Python path is wrong, or the output directory is not writable.
- **Resolve:** confirm the scheduled account resolves ADC (or a secured credential), use absolute paths for the interpreter and `--out`, and check the tool's exit code/log. Re-render the entry with `gwsdk_schedule_config_helper.py` and confirm the minimal scope is on the DWD grant.

Best Practices

Durable practices for building and operating Admin SDK / API automation on Google Workspace.

Security & credentials

- **Never put keys or secrets in code.** Use ADC, Workload Identity, or a secrets manager. Do not create or download service-account JSON keys unless unavoidable; if you must, store them outside the repo and rotate on a schedule.
- **Least scope, always.** Grant each task only the narrowest OAuth scope it needs; default to `*.readonly`. Add a write scope to the DWD grant only for the specific tool that needs it, and review the grant whenever a new automation is added.
- **Least-privilege impersonation.** Impersonate a purpose-built admin subject (`Impersonation Subject – Reports – ReadOnly`) with only the privileges the reports require — never a super-admin for routine runs.
- **Treat domain-wide delegation as sensitive.** The DWD grant `DWD Grant – AdminSDK – LeastScope` is a domain-level trust; audit it periodically, keep the scope list minimal and documented, and remove the service account's client ID promptly if the automation is retired.
- **Rotate and revoke.** Rotate credentials on a schedule; revoke immediately on suspected exposure.

Safe automation

- **Dry-run first, apply explicitly.** Every write tool defaults to `--dry-run` and prints the exact plan; a change happens only with `--apply`. Keep that discipline for every new tool.
- **Idempotent by design.** Write automations should compute desired-vs-actual and make only the needed changes, so re-running is a no-op. Test idempotency (Gate 4-B) before trusting a tool.
- **Always log a reversible change record.** Apply runs write a per-field prior-value log enabling `--revert`. Keep those logs — they are your rollback.
- **Pilot before fleet.** Prove every automation on the pilot OU/group, then expand one controlled wave at a time.

Reliable API access

- **Paginate everything.** Directory / Reports / Drive / Groups results are paged; never assume a single page. Use the shared client's pagination.
- **Back off on 429 / 5xx.** Use exponential backoff with jitter and a retry cap. Respect per-project quotas; keep automation in its own project so its quota is isolated.
- **Handle Reports API latency.** Activity events can lag; treat reports as point-in-time evidence and **verify current latency for your event types** in Google's docs before drawing real-time conclusions.

• **Prefer read-only scopes** for anything that only reports.

Reporting & evidence

• **Name and date every artifact** `<Purpose> – <Scope> – <Stage>` with a timestamp, and write both CSV

(for humans) and JSON (for pipelines) where useful. Consistent naming makes evidence auditable.

• **Make reports deterministic** — same inputs, same output ordering — so diffs between runs are

meaningful.

• **Schedule for cadence, not for real-time.** Use Cloud Scheduler / cron / Task Scheduler for recurring

evidence; ensure the scheduled context has ADC available.

Maintainability

• **Centralize auth and the API client** so every tool requests scopes and handles pagination/retry the

same way; a new tool should reuse them, not re-implement them.

• **Keep vendor-specific detail current.** Google console paths, API scopes, and Reports event names

change; verify against current Google documentation and flag preview/tenant-specific behavior "verify in your tenant".

• **Version desired-state inputs** (provisioning CSVs) in change control so provisioning is reviewable.

Deliverables Summary

This solution delivers implementation documentation plus five authentic Python tools that build and operate a least-scope Admin SDK / API automation and reporting toolkit for Google Workspace.

Documentation

• **Solution Overview** — the automation model: secure service identity, least-scope auth, the

Directory/Reports/Drive/Groups APIs, reusable patterns, scheduling, benefits, limitations, and the Assess → Design → Configure → Pilot → Validate → Expand deployment model.

• **Requirements Guide** — solution-specific prerequisites (super-admin for one-time setup, a dedicated

Cloud project, a delegated service identity, no committed keys, a Python 3.10+ toolchain, a least-scope inventory), each with how to verify and how to implement.

• **Implementation Guide** — exact ordered steps to stand up the service identity and least-scope auth,

build authenticated API access, run discovery + reports read-only, dry-run then apply provisioning on a pilot OU, and schedule recurring reports — with pass/fail gates and a documented rollback.

• **Testing & Validation** — eight procedures with expected pass/fail (auth least-scope, discovery

read-only, report accuracy, provisioning idempotency, apply correctness, rollback, no-secrets, and scheduled evidence).

• **Troubleshooting** — common failure modes (auth/delegation errors, quota/429 throttling, Reports API latency, permission and scope errors, pagination gaps) and resolutions.

• **Best Practices** — durable practices for secure credentials, safe automation, reliable API access, auditable reporting, and maintainability.

The 5 Python tools

1. `gwsdk_auth_scope_validator.py` — validates the service identity and least-scope OAuth setup;

lists granted scopes, flags any broader than the task needs, and optionally runs read-only probes. Proves Gate 3-A. **Read-only.**

1. `gwsdk_tenant_discovery.py` — read-only Directory-API discovery producing `Discovery Report –

Tenant – Assess` (user counts, OU distribution, admin/2SV signals, suspended accounts) as dated CSV+JSON. **Read-only.**

1. `gwsdk_provision_users.py` — idempotent user/OU/attribute provisioning that reconciles to a

desired-state CSV; **dry-run by default**, prints the plan, applies only with `--apply`, writes a reversible change log, and supports `--revert`. **Safe-by-default (write only on --apply).**

1. `gwsdk_sharing_audit_report.py` — read-only report generator producing `Sharing Report –

External – Validate `(Drive external exposure) and` Audit Report – Login – Validate` (Reports API login/admin activity) as dated CSV+JSON. **Read-only.**

1. `gwsdk_schedule_config_helper.py` — renders `Schedule Definition – Reports – Expand`: exact

command lines and cron / Cloud Scheduler / Task Scheduler entries plus the minimal scope each job needs. Makes no API calls and embeds no credentials. **Read-only / generative.**

Every tool authenticates via ADC / a secured credential (no keys in code), requests only the scopes it needs, paginates, and backs off on 429/5xx.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

gwsdk_auth_scope_validator.py

gwsdk_provision_users.py

gwsdk_schedule_config_helper.py

gwsdk_sharing_audit_report.py

gwsdk_tenant_discovery.py